ML Model Deployment Strategies and Model Serving using FastAPI

Dataset: Titanic – Machine Learning from Disaster (Kaggle train.csv)

Objective

Train and save a classification pipeline, demonstrate online, batch and streaming inference, and serve predictions through a REST API built with FastAPI.

Workflow

The training script preprocesses numeric and categorical features, splits labelled rows into training and testing portions, trains a Random Forest classifier, measures hold-out accuracy, and saves the complete pipeline with Joblib. The same saved pipeline is reused by all inference scripts and the API.

Observations

1. Online vs Batch Inference. Online inference predicts one incoming record immediately. Batch inference processes a group of records together.

2. Streaming vs Batch Inference. Streaming inference processes records as they arrive, while batch inference waits for a collected group before processing.

3. Why FastAPI? FastAPI provides HTTP endpoints, Pydantic validation, and automatically generated interactive API documentation at /docs.

4. Why reuse one trained model? Reusing the same pipeline keeps preprocessing and model parameters consistent and makes predictions comparable across inference methods.

Evaluation and results

The measured accuracy is generated in metrics.json when train_model.py is run against the official Kaggle train.csv. No accuracy number is stated in this report because the dataset was not attached to the assignment document and the model has not been executed against it in this package.

API tests

GET /health reports whether the model has loaded. POST /predict validates passenger fields and returns a binary prediction. An invalid Pclass (for example 9) or unsupported Sex value should be rejected with HTTP 422 by Pydantic validation.

Dataset source

Kaggle Titanic – Machine Learning from Disaster: https://www.kaggle.com/competitions/titanic/data